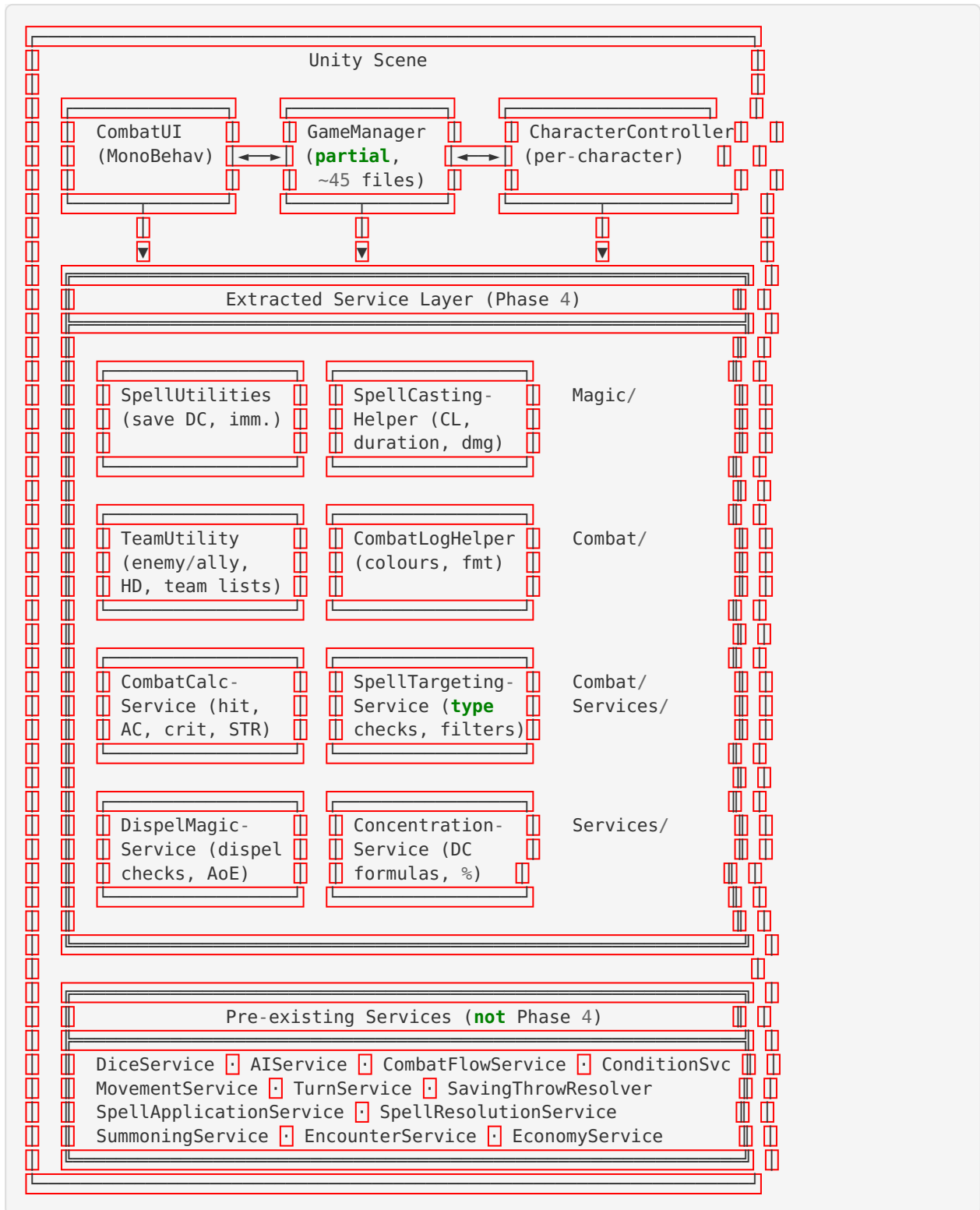


Architecture — Service Layer Overview

Project: D&D 3.5e Prototype (Unity)

Last updated: Phase 4K

High-Level Structure



Directory Layout

```
Assets/Scripts/
├── Magic/
│   ├── SpellUtilities.cs           ← Phase 4A (static, 32 sites)
│   ├── SpellCastingHelper.cs      ← Phase 4B (static, 125 sites)
│   ├── SpellDatabase.cs
│   └── ...
├── Combat/
│   ├── TeamUtility.cs             ← Phase 4C (static, 88 sites)
│   ├── CombatLogHelper.cs         ← Phase 4F-H (static, 51+ sites)
│   ├── CombatCalculationService.cs ← Phase 4J (static, 30+ sites)
│   ├── CombatUtils.cs
│   ├── AttackCalculator.cs
│   └── ...
├── Services/
│   ├── DispelMagicService.cs      ← Phase 4D (MonoBehaviour, 30 sites)
│   ├── ConcentrationService.cs    ← Phase 4E (static, 28 sites)
│   ├── SpellTargetingService.cs   ← Phase 4I (static, 40+ sites)
│   ├── AIService.cs
│   ├── DiceService.cs
│   └── ...
├── Core/
│   ├── GameManager.cs             ← God Object (partial, ~45 files)
│   ├── GameManager.SpellCasting.cs
│   ├── GameManager_Spells_*.cs
│   └── ...
└── Tests/
    ├── Services/
    │   ├── ServiceTestRunner.cs   ← Runs all service tests
    │   ├── SpellUtilitiesTests.cs
    │   ├── SpellCastingHelperTests.cs
    │   ├── TeamUtilityTests.cs
    │   ├── DispelMagicServiceTests.cs
    │   ├── ConcentrationServiceTests.cs
    │   ├── CombatLogHelperTests.cs
    │   ├── SpellTargetingServiceTests.cs
    │   └── CombatCalculationServiceTests.cs
```

Design Principles

1. Static-First

Most services are `public static class` — no instantiation, no state.

This keeps call sites simple (`ServiceName.Method(...)`) and avoids dependency injection complexity in a Unity prototype.

2. Exception: DispelMagicService

Dispel/counterspell logic needs scene callbacks (CombatUI, character lists, cleanup hooks) — so it's a `MonoBehaviour` initialised by `GameManager` with `Func<T>` providers rather than direct references.

3. Return Strings, Not Side Effects

`CombatLogHelper` returns formatted strings. Callers decide when/whether to push them to `CombatUI?.ShowCombatLog(...)`. This keeps the helper testable without a scene.

4. Global Namespace

All files share the global namespace (no `using` imports between project files). This is an existing project convention we follow.

5. Thin Delegation Pattern

Original methods on `GameManager` are preserved as thin one-line delegates to the new services. This prevents breaking the 40+ partial-class files that may still call the old name.

Dependency Flow

```

GameManager → SpellUtilities
             → SpellCastingHelper → SpellUtilities
             → TeamUtility
             → DispelMagicService → DiceService
             → ConcentrationService
             → CombatLogHelper      (no deps)
             → SpellTargetingService → TeamUtility
             → CombatCalculationService (no deps)

CharacterController → CombatCalculationService
                   → TeamUtility
                   → SpellCastingHelper

AIService → TeamUtility
          → SpellTargetingService
  
```

No circular dependencies exist between the extracted services.

Commit History

Commit	Phase	Description
f595e50	4B	SpellCastingHelper extraction
99e0672	4C	TeamUtility extraction
fc95512	4D	DispelMagicService extraction
26f26cd	4E	ConcentrationService extraction
b16aee5	4F	CombatLogHelper creation + 51 sites
be1864a	4G	CombatLogHelper batch 2
8c43782	4H	CombatLogHelper batch 3
062e23d	4I	SpellTargetingService extraction
dd3b39a	4J	CombatCalculationService extraction

Future Work

1. **Continue CombatLogHelper migration** — ~1000 remaining `ShowCombatLog` calls can be converted incrementally
2. **Extract SpellEffectService** — spell-application logic still in `GameManager.SpellCasting.cs` (~7000 lines)
3. **Extract TurnManager** — turn/initiative logic from `GameManager`
4. **Add CharacterController mock** — enables full unit testing of services that take `CharacterController` parameters
5. **Consider namespace introduction** — as the service layer grows, namespaces would reduce global-scope pollution